

```
typedef struct Node          // Declare structure of node
{
    int info;
    struct Node *llink;
    struct Node *rlink;
} Node;
```

แต่ละ Node ประกอบด้วย info = เก็บข้อมูล llink = ชี้ไปยัง Node ก่อนหน้า rlink = ชี้ไปยัง Node ถัดไป

```
Node *Allocate() // Allocate 1 node from storage pool
{
    struct Node *temp;
    temp = (Node *)malloc(sizeof(Node)); // Allocate node by
size declare
    return (temp);
}
```

ใช้ malloc() เพื่อสร้าง Node ใหม่ใน Heap Memory

```
void CreateNNode(int n) // Create N Node put data and link it
{
    int i, temp;
    H = NULL;
    H1 = NULL;
    for (i = 1; i <= n; i++) // Count N Node
    {
        p = Allocate();          // Allocate Node
        temp = 1 + rand() % 99; // random difference number
1..99
        p->info = temp;          // Put random data in to node
        if (i == 1)
            H = p; // Set H point to first node
        else
            H1->rlink = p; // Link first node to second node
        p->llink = H1;
        H1 = p;                // Let H1 point to last node
        H1->rlink = NULL; // Set rlink of H1 to NULL
    }
}
```

ภายในจะสลับข้อมูล 1-99 แล้วเชื่อมโยง Node เข้าด้วยกัน H1->rlink = p; 2 p->llink = H1;
 สมมุติสลับได้ 65 12 89 34 28 โครงสร้างจะมีลักษณะดังนี้

NULL <- [65] <-> [12] <-> [89] <-> [34] <-> [28] -> NULL

```
void ShowAllNode()
{
    printf("H = %x\n", H); // Display address of pointer H
    p = H;                // Set start point of p pointer
    i = 1;                // set start value of counter
    while (p != NULL)     // While if NOT NULL
    {
        printf("%d) : %x\t", i, p);        // Show COUNTER and
POINTER
        printf("LLINK : %x\t", p->llink); // Show lLINK
        printf("INFO : %d\t", p->info);   // Show INFO
        printf("RLINK : %x\n", p->rlink); // Show RLINK
        p = p->rlink;                     // Skip to next node
        i++;                             // Skip Counter
    } // End While
} // Enf Fn.
```

แสดงข้อมูล ADDRESS LLINK INFO RLINK ตัวอย่างผลลัพธ์เช่น

H = 1000

1): 1000	LLINK : 0000	INFO : 65	RLINK : 1010
2): 1010	LLINK : 1000	INFO : 12	RLINK : 1020
3): 1020	LLINK : 1010	INFO : 89	RLINK : 1030
4): 1030	LLINK : 1020	INFO : 34	RLINK : 1040
5): 1040	LLINK : 1030	INFO : 28	RLINK : 0000

```
void InsertAfter(int data1)
{
    int temp; // Temporary variable
    if (H == NULL)
        printf("Linked List have no node!!..\n");
    else
    {
        H1 = H;                // Let H1 point at start node
        while (H1 != NULL) // Search for the data while
H1<>NULL
    {
```

```

        if (H1->info == data1) // if Found
        {
            p = Allocate();          // Allocate one
node from storage pool
            printf("\nInsert data : "); // Input data for
insert
            scanf("%d", &temp);      // Read from KBD
            p->info = temp;           // Entry temporary
data into INFO of node
            if (H1->rlink == NULL)
            {
                p->rlink = NULL;
            }
            else
            {
                p->rlink = H1->rlink; // Change pointer
1st for insert node (FAR to NEAR)
                H1->rlink->llink=p; //LLINK(RLINK(H1))=p
            }
            p->llink = H1; // LLINK(P)=H1
            H1->rlink = p; // RLINK(H1)=p
        } // End if
        H1 = H1->rlink; // Skip H1 to next node
    } // End while
} // End IF
} // End Fn.

```

สมมุติข้อมูลเดิมลักษณะดังนี้คือ 10 <-> 20 <-> 30 เมื่อเลือกเมนู Insert After เลือก 20
 ป้อนข้อมูล Insert data = 25 ผลลัพธ์ที่ได้จะเป็น 10 <-> 15 <-> 20 <-> 30

```

void InsertBefore(int data1)
{
    int temp; // Temporary variable
    if (H == NULL)
        printf("Linked List have no node!!..\n");
    else
    {
        H1 = H;          // Let H1 point at start node
        while (H1 != NULL) // Search for the data while
H1<>NULL
        {

```

```

        if (H1->info == data1) // if Found
        {
            p = Allocate();          // Allocate one
node from storage pool
            printf("\nInsert data : "); // Input data for
insert
            scanf("%d", &temp);      // Read from KBD
            p->info = temp;           // Entry temporary
data into INFO of node
            if (H1->llink == NULL)
            {
                p->llink = NULL;
                H = p;
            }
            else
            {
                H1->llink->rlink = p; //
RLINK(LLINK(H1))=p
                p->llink = H1->llink; // LLINK(p)=LLINK(H1)
            }
            H1->llink = p; // LLINK(H1)=p
            p->rlink = H1; // RLINK(P)=H1
        } // End if
        H1 = H1->rlink; // Skip H1 to next node
    } // End while
} // End IF
} // End Fn.

```

สมมุติข้อมูลเดิมลักษณะดังนี้คือ 10 <-> 20 <-> 30 เมื่อเลือกเมนู Insert Before ป้อน 20
เลือกป้อนข้อมูลที่จะเข้าไปใส่ข้อมูล Insert data = 15 ผลลัพธ์จะเป็น 10 <-> 15 <-> 20 <-> 30

```

void DeleteBefore(int data1)
{
    int temp; // Temporary variable
    if (H == NULL)
        printf("Linked List have NO NODE!!..\n");
    else
    {
        H1 = H;          // Let H1 point at start node
        while (H1 != NULL) // Search for the data while
H1<>NULL

```

```

    {
        if (H1->info == data1) // if Found
        {
            if (H1->llink == NULL) // If no more node
                printf("No more node from here,Can't
delete it!!!\n");
            else
            {
                p = H1->llink;          // Mark at node for
Delete
node
                if (p->llink == NULL) // If p is first
                {
                    H1->llink = NULL; // Set link of H1 to
NULL
                    H = H1;           // Set H to first
NODE
                }
                else
                {
                    H1->llink = p->llink; // If not set
link of H1 point same address of p
                    p->llink->rlink = H1;
                }
                free(p); // Free node to storage pool
            } // End if2
        } // End if1
        H1 = H1->rlink; // Skip H1 to next node
    } // End while
} // End IF
} // End Fn.

```

ตัวอย่างข้อมูลปัจจุบันคือ 10 <-> 20 <-> 30 <-> 40 เลือกใช้คำสั่ง Delete Before เลือก 30 Node ที่จะถูกลบคือ 20 ซึ่งจะมีลักษณะผลลัพธ์ดังนี้คือ 10 <-> 30 <-> 40

```

void DeleteSelf(int data1)
{
    int temp; // Temporary variable
    if (H == NULL)
        printf("Linked List have NO NODE!!..\n");
    else

```

```

{
    H1 = H;          // Let H1 point at start node
    while (H1 != NULL) // Search for the data while
H1<>NULL
    {
        if (H1->info == data1) // if Found
        {
            p = H1;          //
Mark at node for Delete
            if (p->llink == NULL && p->rlink == NULL) //
If only one node
                H = NULL;
            else
            {
                if (p->llink == NULL) // Check if first
NODE
                {
                    H = p->rlink;    // Let H point to
RLINK(P)
                    H->llink = NULL; // Assign
LLINK(H)=NULL
                }
                else
                {
                    if (p->rlink == NULL) // Check if Last
NODE
                    {
                        p->llink->rlink = NULL;
                    }
                    else // NORMAL
                    {
                        p->llink->rlink = p->rlink;
                        p->rlink->llink = p->llink;
                    }
                }
            }
            free(p); // Free node to storage pool
        } // End if1
        H1 = H1->rlink; // Skip H1 to next node
    } // End while
} // End IF
} // End Fn.

```

ตัวอย่างข้อมูลปัจจุบัน 10 <-> 20 <-> 30 <-> 40 เลือกใช้คำสั่ง DeleteSelf เลือก 30

จะทำการลบข้อมูลตัวมันเองที่ผู้ใช้ป้อนข้อมูลซึ่งมีผลลัพธ์ดังนี้ 10 <-> 20 <-> 40 โปรแกรมจะเชื่อม
p->llink->rlink = p->rlink;
p->rlink->llink = p->llink;
แล้ว free(p); เป็นการคืนหน่วยความจำ

```
void DeleteAfter(int data1)
{
    int temp; // Temporary variable
    if (H == NULL)
        printf("Linked List have NO NODE!!..\n");
    else
    {
        H1 = H; // Let H1 point at start node
        while (H1 != NULL) // Search for the data while
H1<>NULL
        {
            if (H1->info == data1) // if Found
            {
                if (H1->rlink == NULL) // If no more node
                    printf("No more node from here,Can't
delete it!!!\n");
                else
                {
                    p = H1->rlink; // Mark at node for
Delete
                    if (p->rlink == NULL) // If p is last node
                        H1->rlink = NULL; // Set link of H1 to
NULL
                    else
                    {
                        H1->rlink = p->rlink; // If not set
link of H1 point same address of p
                        p->rlink->llink = H1;
                    }
                    free(p); // Free node to storage pool
                } // End if2
            } // End if1
            H1 = H1->rlink; // Skip H1 to next node
        } // End while
    } // End IF
} // End Fn.
```

ตัวอย่างข้อมูล 10 <-> 20 <-> 30 <-> 40 เลือกใช้คำสั่ง Delete After เลือก 20
Node ข้อมูลที่จะถูกลบคือ 30 ซึ่งจะมีลักษณะดังนี้ 10 <-> 20 <-> 40

```
int main() // MAIN Fn.
{
    n = 10;          // Set amount of node
    CreateNNode(n); // Call Fn. Create N nodes
    printf("PROGRAM DOUBLY LINKED LIST \n");
    printf("===== \n");
    printf("All Data in Linked List \n");
    ShowAllNode(); // Call Fn. Show all node
    ch = ' ';
    while (ch != 'E' && ch != 'e')
    {
        printf("MENU>> [B:InsertBefore A:InsertAfter\n");
        printf(" O:DeleteBefore X:Delete itself\n");
        printf(" D:DeleteAfter E:Exit]\n");
        scanf(" %c", &ch); // GCC/MinGW not working for
conio.h -> getch()
        switch (ch)
        {
            case 'b':
            case 'B':
                printf("\nInsert After data : "); // Input data
for insert before
                scanf("%d", &data);
                InsertBefore(data); // Call Fn. Insert after data
                printf("\nAll Data in Linked List AFTER
INSERTED\n");
                ShowAllNode(); // Call Fn. Show all node
                break;
            case 'a':
            case 'A':
                printf("\nInsert After data : "); // Input data
for insert after
                scanf("%d", &data);
                InsertAfter(data); // Call Fn. Insert after data
                printf("\nAll Data in Linked List AFTER
INSERTED\n");
                ShowAllNode(); // Call Fn. Show all node
                break;
            case 'o':
```



```

        case 'O':
            printf("\nDelete Before data : "); // Input data
for Delete after
            scanf("%d", &data);
            DeleteBefore(data); // Call Fn. Delete after data
            printf("\nAll Data in Linked List AFTER
DELETED\n");
            ShowAllNode(); // Call Fn. Show all node
            break;
        case 'x':
        case 'X':
            printf("\nDelete ItSelf data : "); // Input data
for Delete after
            scanf("%d", &data);
            DeleteSelf(data); // Call Fn. Delete after data
            printf("\nAll Data in Linked List ITSELF
DELETED\n");
            ShowAllNode(); // Call Fn. Show all node
            break;
        case 'd':
        case 'D':
            printf("\nDelete After data : "); // Input data
for Delete after
            scanf("%d", &data);
            DeleteAfter(data); // Call Fn. Delete after data
            printf("\nAll Data in Linked List AFTER
DELETED\n");
            ShowAllNode(); // Call Fn. Show all node
            break;
    } // End Switch...case
} // End While */
getch(); // Wait for KBD hit
} // End MAIN

```

สร้าง Node จำนวน 10 Nodes จากนั้นแสดงข้อมูลเริ่มต้น แล้วเข้าสู่เมนู

```
PS D:\Bachelor's\ENGCE124\lab6> ./6-1
PROGRAM DOUBLY LINKED LIST
=====
All Data in Linked List
H = 9b5769d0
1) : 9b5769d0  LLINK : 0      INFO : 42      RLINK : 9b5769f0
2) : 9b5769f0  LLINK : 9b5769d0  INFO : 54      RLINK : 9b576a10
3) : 9b576a10  LLINK : 9b5769f0  INFO : 98      RLINK : 9b576a30
4) : 9b576a30  LLINK : 9b576a10  INFO : 68      RLINK : 9b576a50
5) : 9b576a50  LLINK : 9b576a30  INFO : 63      RLINK : 9b576a70
6) : 9b576a70  LLINK : 9b576a50  INFO : 83      RLINK : 9b576a90
7) : 9b576a90  LLINK : 9b576a70  INFO : 94      RLINK : 9b576ab0
8) : 9b576ab0  LLINK : 9b576a90  INFO : 55      RLINK : 9b576ad0
9) : 9b576ad0  LLINK : 9b576ab0  INFO : 35      RLINK : 9b576af0
10) : 9b576af0 LLINK : 9b576ad0  INFO : 12      RLINK : 0
MENU>> [B:InsertBefore A:InsertAfter
O:DeleteBefore X:Delete itself
D:DeleteAfter E:Exit]
```

ผู้ใช้สามารถเลือกคำสั่งได้เรื่อย ๆ จนป้อนข้อมูลปุ่ม “E” เพื่อออกจากโปรแกรม

สรุปผลการทดลอง

- โปรแกรมสามารถสร้าง Doubly Linked List ได้สำเร็จโดยใช้ Dynamic Memory Allocation (malloc) ในการสร้าง Node แต่ละ Node
 - มี Pointer 2 ตัว คือ llink และ rlink ทำให้สามารถเดินข้อมูลได้ทั้งไปข้างหน้าและย้อนกลับ
 - สามารถแทรกข้อมูลก่อนและหลัง Node ที่กำหนดได้อย่างถูกต้อง
 - สามารถลบ Node ก่อนหน้า ลบ Node ปัจจุบัน และลบ Node ถัดไปได้ การลบ Node ใช้ free() คำนวณหน่วยความจำ ช่วยลดการสูญเสียหน่วยความจำ (Memory Leak)
 - ฟังก์ชัน ShowAllNode() แสดง Address ของ Node และการเชื่อมโยงของ Pointer ทำให้เห็นโครงสร้างของ Doubly Linked List อย่างชัดเจน
 - โปรแกรมแสดงให้เห็นหลักการสำคัญของโครงสร้างข้อมูลแบบ Linked List ได้แก่ การสร้าง การเชื่อมโยง การแทรก และการลบข้อมูลแบบ Dynamic Data Structure
- ซึ่งมีความยืดหยุ่นกว่าการใช้ Array
- เนื่องจากไม่จำเป็นต้องกำหนดขนาดล่วงหน้าและสามารถเพิ่มหรือลบข้อมูลระหว่างการทำงานได้อย่างมีประสิทธิภาพ.